NLP
NATURAL LANGUAGE PROCESSING

It is a branch of AI. It enables computers to read, understand, write & communicate in human language.

NLP Bridges the gap between human language & computer language (0s & 1s).

↳ Applications of NLP

• Chatbots & Virtual Assistants

understand user input & generate meaningful responses.

Ex - ChatGPT, siri, Gemini, claude, Qwen etc

• Email spam Filtering

Detects spam emails using suspicious words or phrases.

Ex - "you won a lottery" - that indicates that's a spam

• sentiment Analysis

Analyzes customer reviews, tweets, comments & feedback.

It classifies opinions as:

* Positive
* Negative
* Neutral

→ Rule-Based NLP (old-NLP)

Uses Manually written grammer rules & if-else conditions to process & understand text.

↳ Uses

- spam filtering
- keyword matching
- simple chatbots
- command-based systems

↳ Example

if email contains "lottery" mark as spam

Ex- Bruh, you got 10K lottery on xyz.app

output - spam

↳ Disadvantages

- Not scalable (language constantly changes)
- cannot understand context like - sarcasm

ex- "Bruh I love to work 18 hrs a day what a life"

It understand as appericeation from "love" "what" "like" that a sarcasm he says in anger.

- cannot detect sarcasm or ambiguity
- Requires manual rule writing

→ Statistical (Machine Learning NLP)

Uses machine learning algorithms to learn language patterns from data instead of relying on manually written rules.

↳ Uses :

- sentiment Analysis
- spam Detection
- Text classification
- Language translation.

↳ ML Models Used

- Naive Bayes
- Logistic Regression
- support Vector Machine (SVM)

↳ Example

Model ~~get~~ trained on positive & Negative movie reviews

Ex- This movie was amazing

Output = Positive

while focusing on amazing it predicts Positive review.

↳ Disadvantages

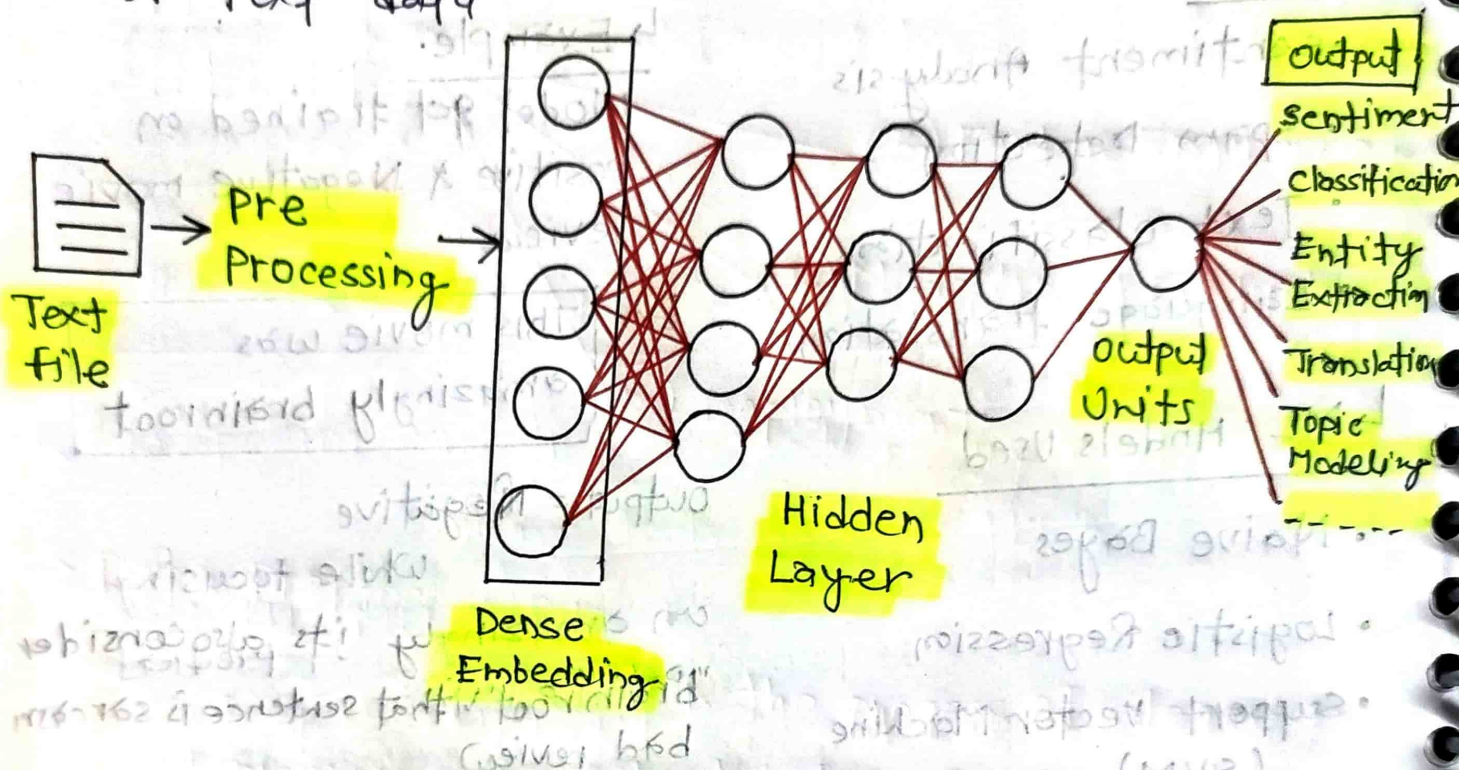
- Requires training data
- Needs more computation
- Traditional ML struggles with context & sarcasm

Ex- This movie was amazingly I slept After 10 minutes

Output - Positive, But it was a sarcasm Negative review.

→ Deep learning NLP (Modern NLP)

It uses Neural Network to automatically learn language patterns, context & meaning from large amounts of text data.



↳ common deep learning Models

- RNN (Recurrent Neural Network)
- LSTM (Long short Term Memory)
- GRU (gated Recurrent Unit)
- Transformer (Modern standard)
- BERT
- GPT

↳ Example : Review - "This movie was amazing...I slept after 10 minutes.

Output - Negative

Unlike traditional ML, deep learning models consider the entire sentence & context, so they recognize that the review is sarcastic & actually expres negative opinion.

↳ Uses

- Machine Translation (google Translate)
- chatbots & virtual assistants
- Text summarization
- sentiment Analysis
- Question Answering
- Text Generation (chatGPT)

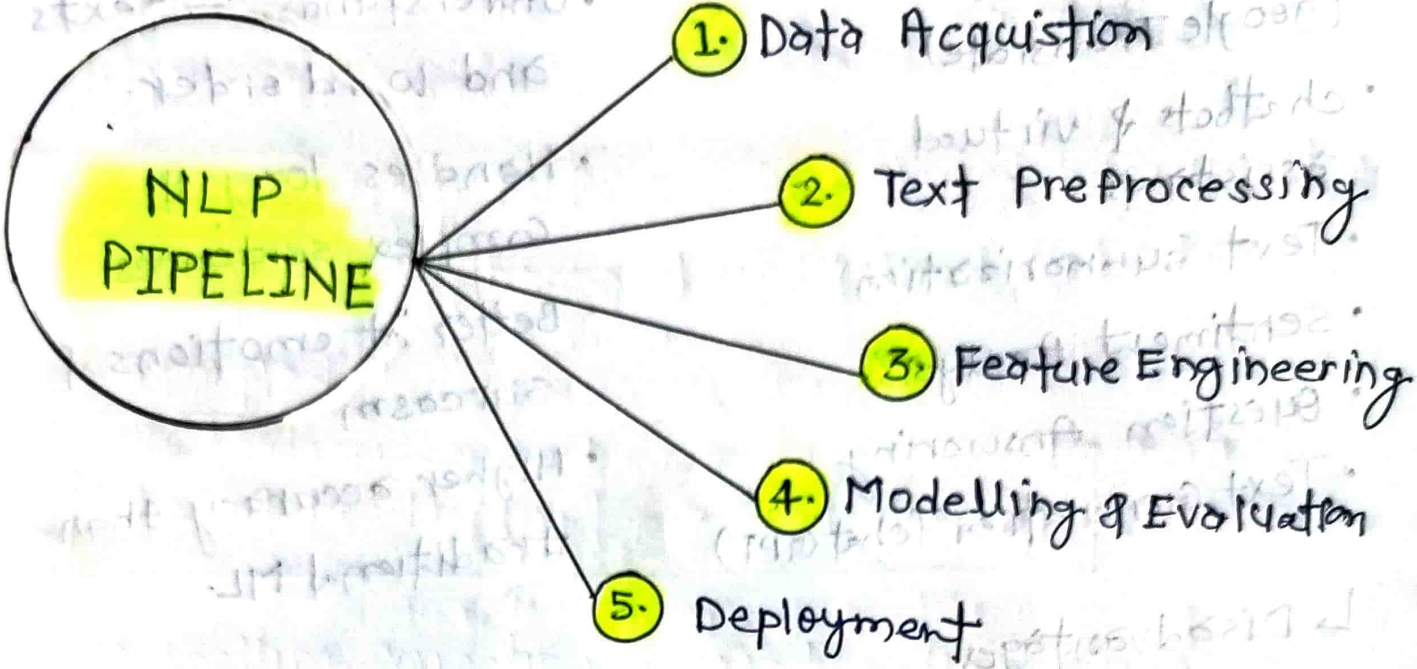
↳ Advantages

- Understands contexts and word order.
- Handles long & complex sentences
- Better at emotions & sarcasm
- Higher accuracy than traditional ML.

↳ Disadvantages

- Requires large datasets
- Needs powerful GPU/TPU
- Longer Training time
- More computationally expensive.

→ NLP Pipeline



→ Data Preprocessing

It is the process of cleaning & preparing raw text before feeding it into a machine learning or deep learning model to improve performance & accuracy.

→ Data Preprocessing Steps

↳ **(1.) Lowercasing**: convert all text to lowercase.

Ex- "The Cat" → "the cat"

↳ **(2.) Remove Punctuation**: Remove symbols like - , . ! ? #

Traditional ML NLP usually removed but In Deep learning NLP usually not removed because of punctuation helps understand context

Ex- "Hello!!!" → "Hello"

↳ (3.) Remove Numbers : Remove digits if they are not important for the task.

- old ML NLP : optional
- Deep learning NLP : Usually kept if they contain they contain useful Information.

ex- I have 2 dogs → I have dogs

↳ (4.) Remove URL & HTML Tags : Remove website links & html tags.

- ML NLP : used
- Deep Learning NLP : used

Ex- Visit https://thik_ba.com Hello
↓
Visit Hello

↳ (5.) Remove Emojis & special characters

Remove emoji 😊 & unnecessary symbol.

- ML NLP : used
- Deep learning NLP : emoji are usually kept because they carry emotion. Remove only unnessary special character.

Ex- Ka hall hai #!! 😊😞 → Ka hall hai → ML NLP
Ka hall hai 😊😞 → DL NLP

↳ **(6.) Remove stop words** : Remove common words like like 's, the, are, and

- ML NLP : used
- Deep Learning NLP: usually not used because stop words help the model understand context.

- **Example-**

"The cat is sleeping" → "cat sleeping"

↳ **(7.) spelling correction** : correct spelling mistakes using tools like Textblob

- ML NLP - optional
- Deep Learning NLP - optional But used only when the dataset has many spelling errors.

- **Example**

"Mahinae Learnng" → Machine Learning

↳ **(8.) Tokenization** : split text into smaller units (tokens/words)

- ML NLP : Used
- DL NLP : Used

- **Example**

"I am Aniket" → ['I', 'am', 'Aniket']

↳ (g.) stemming / Lemmatization converts word to their root or base form

- ML NLP : Commonly used
- DL NLP : Usually not used because modern models learn word meaning & context directly.

- Example

Running	→ Run
studies	→ study

PROJECT : 1
EMOTION DETECTION USING ML (NLP)

↳ overview

- Devdoped an Emotion Detection system using Traditional NLP & Machine Learning.
- performed text preprocessing (lowercasing, punctuation removal, number removal, emoji removal, & stopword removal
- converted text into numerical features using Bag of words (BoW) & TF-IDF.
- Trained Naive Bayes & Logistic Regression models for emotion classification.
- Evaluate model performance using accuracy score
- Compare different feature extraction technique & machine learning algorithms to identify the better performing approach

1. IMPORTS LIBRARIES

In[1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

2. LOAD DATASET

In [2]:

```
df = pd.read_csv('train.txt',
                 sep = ';' #columns seperated by ';'
                 header = None #Dataset has no header
                 names = ['text','emotion'])
                 #Assign column names

df.head()
df.isnull().sum()
```

3. ENCODE EMOTION LABELS

In [3]:

```
unique_emotions = df['emotion'].unique()
#geeting all unique emotion labels
emotion_numbers = { }
#create an empty dictionary
i = 0
# initialize label no.
for emo in unique_emotions:
#loop through each emotion
    emotion_numbers[emo] = i #Assign numerical label
    i += 1    #Increase label number

df['emotion'] = df['emotion'].map(emotion_numbers)
#Replace emotion names with numbers
df
```

4. CONVERT TEXT TO LOWERCASE & REMOVE PUNCTUATION

In [4].

```
df['text'] = df['text'].apply(lambda x: x.lower())
# Convert every sentence into lowercase
import string  # for punctuation removal & character filtering
def remove_punc(text):
#Function to remove punctuation
    return text.translate(
        str.maketrans('', '', string.punctuation))
df['text'] = df['text'].apply(remove_punc)
#Apply punctuation removal
```

5. REMOVE NUMBERS

```
def remove_numbers(text):
#Function to remove digit
    new = ""  # store cleaned text
    for i in text: # iterate through each character
        if not i.isdigit(): #keep only non-digit
            new += i          characters
    return new
df['text']=df['text'].apply(remove_numbers)
#Apply number approval
```

6. REMOVE EMOJIS

In[6]:

```
def remove_emojis(text):
    new = "" #stored cleaned text
    for i in text:
        if i.isascii(): #keep only ASCII characters
            new += i
    return new
df['text'] = df['text'].apply(remove_emojis)
# Apply emoji removal
```

7. IMPORT NLTK LIBRARIES & DOWNLOAD RESOURCES

In[7]

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download('punkt')
nltk.download('stopwords')
```

8. LOAD & REMOVE STOPWORDS

In[8]:

```
stop_words = set(stopwords.words('english'))
#load ENGLISH stopwords
df.loc[1]['text'] #Display sample text before cleaning
def remove(txt):
    words = txt.split() #split sentence into words
    cleaned = [] #store filtered words
    for i in words:
        if i not in stop_words: #keep meaningful words
            cleaned.append(i)
    return ' '.join(cleaned) #Join words into a sentence
df['text'] = df['text'].apply(remove) # Apply stopword removal
```

9. DISPLAY CLEANED DATASET

In [9]:

```
df.loc[1]['text']
#Displat cleaned text sample
df.head()
```

→ Embedding

- convert words into dense numerical vectors.
- captures meaning, context & semantic relationships
- used in Deep learning NLP.

↳ Example :

- word2Vec
- GloVe
- FastText

↳ Ex

King ↔ Queen

Man ↔ women

The vectors are mathematically close because their meaning are related.

→ Feature Extraction/Vectorization

ML & DL model cannot Understand raw text directly. Text must first be converted into numerical vectors.

↳ Traditional Feature Extraction

- converts text into numbers using statiscal methods.
- Based on word frequency.
- Does not Understand context or semantics

↳ Techniques

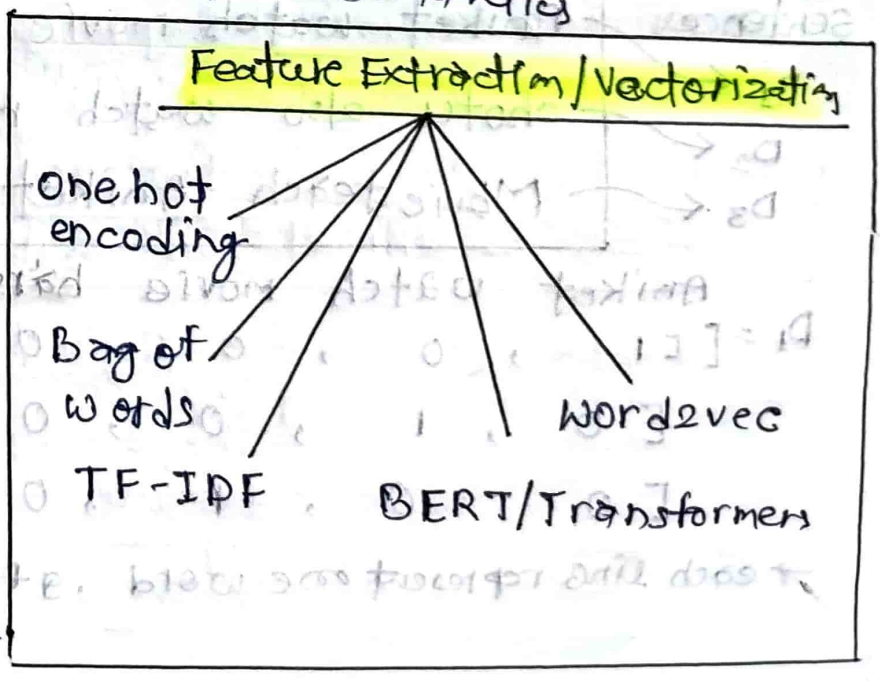
- One hot Encoding
- Bag of words (BoW)
- TF-IDF

↳ Ex

King → [1, 0, 0]

Queen → [0, 1, 0]

The model treats King & Queen as completly unrelated.



→ one-hot encoding

It converts every word into a binary vector where only one position is 1 and all others are 0.

↳ Some Terms

- **corpus**: complete collection of text documents.
- **Documents**: A single sentence, paragraph, or article.
- **Vocabulary**: List of all unique words in the corpus.

↳ Advantages

- Easy to Understand
- Easy to Implement

↳ Disadvantages

- Very sparse vectors
- Large memory usage
- cannot handle new (oov) words
- different document sizes
- No semantic meaning

↳ Example

Sentences

D_1 ← Aniket watch movie

D_2 ← chotu also watch movie

D_3 ← Movie teach Aniket

Aniket watch movie chotu also teach

$D_1 = [[1, 0, 0, 0, 0, 0],$
$[0, 1, 0, 0, 0, 0],$
$[0, 0, 1, 0, 0, 0]]$

→ each line represent one word, if position match then 1.

$D_2 = [[0, 0, 0, 1, 0, 0]$
$[0, 0, 0, 0, 1, 0]$,
$[0, 1, 0, 0, 0, 0]$,
$[0, 0, 1, 0, 0, 0]]$

same goes for D_3, O/P $D_1 = 1$, $D_2 = 1$, $D_3 = 0$

→ Bag of words (BoW)

It converts each document into a vector based on word frequency (How much time it repeated)

↳ Advantages

- Easy to implement
- Fixed-size vectors
- Fast
- Work well with ML Models

↳ Working

(1.) create Vocabulary
(2.) Count frequency of each word
(3) Ignore word order

↳ Disadvantages

- Sparse vectors
- OOV problem
- Ignore word order
- Limited semantic understanding
- orderless

↳ Ex- same Sentence

Aniket watch movie chotu also read

$D_1 = [1, 1, 1, 0, 0, 0]$
$D_2 = [0, 1, 1, 1, 1, 0]$
$D_3 = [0, 0, 2, 1, 0, 0, 1]$

O/P = $D_1 = 1$
$D_2 = 1$
$D_3 = 0$

→ CountVectorizer Parameters

↳ (1.) max-features : Limit vocabulary size to the most frequency words.

- ex - `countVectorizer(max-features = 5000)`

↳ (2.) binary = True : Stores only whether word is present.

→ N-grams

These are groups of N consecutive words/ adjact neighbouring words as a single feature to capture word order & local context

↳ Types

(1.) Unigram (1 word) : Represents each individual word as a seprate feature

Ex - I , am , aniket

(2.) Bigram (2 words) : Represents every pair of adjacent words as a single feature.

Ex - I aniket , am aniket

(3.) Trigram (3-words) : Represents every group of three adjacent words as a single feature.

Ex - I am Aniket

↳ Advantages

- understand little bit semantic meaning
- captures word order & local context

↳ disadvantages

- Increases vocabulary
- produces sparse vector with many zeros
- OOV problem (out of vocabulary)
- Higher memory usage & computational cost than Unigrams.

→ TF-IDF

It assigns higher weight to important words and lower weight to common words.

- Formula

$$TF\text{-}IDF = TF \times IDF$$

↳ **Term Frequency (TF)** : Measures how often a word appears in a document.

- formula

$$TF = \frac{(\text{No. of occurance of term in document})}{(\text{Total no. of terms in Doc})}$$

↳ **Inverse Document Frequency (IDF)** : Measures how rare a word is across all documents.

- Formula

$$IDF = \log_e \frac{(\text{Total no. of doc in corpus})}{(\text{number of doc with containing word})}$$

↳ Advantages

- Gives importance to meaningful words
- Better than BoW for search & document ranking
- Reduces influence of common words

↳ Disadvantages

- sparse vector
- OOV problem
- High dimensionality
- Still doesn't understand context or semantics

↳ Example

Common word : **the**

Low TF-IDF score.

Rare word : **transformer**

High TF-IDF score.

PROJECT 1
EMOTION DETECTION USING ML(NLP)

10. TRAIN TEST SPLIT & IMPORT FEATURE EXTRACTION

In [10]:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    df['text'], #Input text data
    df['emotion'], Target emotion labels
    test_size=0.20, random_state=42)
from sklearn.feature_extraction.txt import
CountVectorizer, TfidVectorizer
# count Vector → converts text into Bag of words (BoW)
                 vectors
#TfidFVectorizer → converts text into TF-IDf vectors.
from sklearn.naive_bayes import MultinomialNB
# It is Multinomial Naive bayes Algorithm
from sklearn.metrics import accuracy_score
```

11. BAG OF WORDS (BoW) FEATURE EXTRACTION

In [11]:

```
bow_vectorizer = CountVectorizer()
# create of Bag of word vectorizer
X_train_bow = bow_vectorizer.fit_transform(X_train)
X_test_bow = bow_vectorizer.transform(X_test)
```

12. TRAIN NAIVE BAYES USING BOW & PREDICTION

In [12]:

```
nb_model = MultinomialNB()
# create the Multinomial Naive Bayes Model
nb_model.fit(X_train_bow, y_train)
#Train the model using BoW features
pred_bow = nb_model.predict(X_test_bow)
print(accuracy_score(y_test, pred_bow))
pred_bow
```

13. TF-IDF FEATURE EXTRACTION & TRAIN NAIVE BAYES

In [13]:

```
tfidf_vectorizer = TfidfVectorizer()
# create a TF-IDF vectorizer
X_train_tfidf = tfidf_vectorizer.fit_transform(
                X_train)

X_train_tfidf = tfidf_vectorizer.transform(X_test)
nb2_model = MultinomialNB()
# create another multinomial Naive Bayes Model
nb2_model.fit(X_train_tfidf, y_train)
```

14. PREDICT & EVALUATE (TF-IDF + NAIVE BYES

In [14]

```
y_pred = nb2_model.predict(X_test_tfidf)
print(accuracy_score(y_test, ypred)
```

15. LOGISTIC REGRESSION

In [15]:

```
from sklearn.linear_model import LogisticRegression
logistic_model = LogisticRegression(max_iter=1000)
# create logistic Regression Model
logistic_model.fit(X_train_tfidf, y_train)
# Train Logistic Regression using TF-IDF features
log_pred = logistic_model.predict(X_test_tfidf)
print(accuracy_score)(y_test, log_pred))
```

Final Result comparison

- Naive Bayes + Bag of words → 96.81 %.
- Naive Bayes + TF-IDF → 66.09 %.
- Logistic + TF-IDF → 86.2 % (Best Model)